

Lab2 物理内存和页表

窦楷然 2210652 屈华晨 2210650 张梓恒 2211279

报告要求

- 基于markdown格式来完成，以文本方式为主
- 填写各个基本练习中要求完成的报告内容
- 列出你认为本实验中重要的知识点，以及与对应的OS原理中的知识点，并简要说明你对二者的含义，关系，差异等方面的理解（也可能出现实验中的知识点没有对应的原理知识点）
- 列出你认为OS原理中很重要，但在实验中没有对应上的知识点

练习1

练习1：理解first-fit 连续物理内存分配算法（思考题） first-fit 连续物理内存分配算法作为物理内存分配一个很基础的方法，需要同学们理解它的实现过程。请大家仔细阅读实验手册的教程并结合kern/mm/default_pmm.c中的相关代码，认真分析default_init，default_init_memmap，default_alloc_pages， default_free_pages等相关函数，并描述程序在进行物理内存分配的过程以及各个函数的作用。请在实验报告中简要说明你的设计实现过程。

请回答如下问题： 你的first fit算法是否有进一步的改进空间？

以下是更加详细的分析，每个函数根据实际流程拆分成多个关键代码段，并附加更详细的解释，帮助理解 first-fit 算法在物理内存分配中的实现细节。

代码及详细分析

default_init：初始化内存分配器

```
static void default_init(void) {  
    list_init(&free_list);  
    nr_free = 0; // 空闲页数  
}
```

功能描述：

default_init 是初始化函数，用于设置物理内存分配器的初始状态：

1. list_init(&free_list); - 初始化空闲页列表 free_list，将其设为空。
2. nr_free = 0; - 初始化空闲页计数器 nr_free 为0，表示当前没有空闲页。

作用分析：

此函数确保分配器在最初没有任何可用的内存块。它主要用于系统启动或重置内存分配时，将分配器的状态初始化为干净状态。

default_init_memmap：初始化内存块

`default_init_memmap` 将内存页块初始化为空闲块，并加入到空闲列表中。

第1部分：初始化每个页面的状态

```
for (struct Page *p = base; p != base + n; p++) {
    assert(PageReserved(p));           // 确保页面最初是已保留状态
    p->flags = p->property = 0;         // 重置页面标志和属性
    set_page_ref(p, 0);                // 将引用计数设为0
}
```

功能描述：

该部分循环遍历每个页面，将它们的状态重置为“空闲”：

- `assert(PageReserved(p));` - 检查页面初始状态是否为保留状态。
- `p->flags = p->property = 0;` - 清除标志位和属性字段。
- `set_page_ref(p, 0);` - 设置引用计数为0。

作用分析：

这一过程确保该内存块中的页面可用于分配。标志位和属性字段的清除可以防止该块因残留标志而被误判为已占用。

第2部分：设置空闲块的起始页面

```
base->property = n;                    // 设置空闲块大小
SetPageProperty(base);                // 设置此页面为空闲块的起始页面
nr_free += n;                         // 更新空闲页面数量
```

功能描述：

设置内存块的第一个页面为空闲块的标识页面，并记录块大小：

- `base->property = n;` - 记录空闲块大小 `n`。
- `SetPageProperty(base);` - 将 `base` 页面标记为空闲块的起始页面。
- `nr_free += n;` - 更新系统中的总空闲页数。

作用分析：

此部分通过在块的起始页面记录块大小和属性，确保后续分配时可以快速判断块大小。

第3部分：将块插入空闲列表的合适位置

```

if (list_empty(&free_list)) {
    list_add(&free_list, &(base->page_link));
} else {
    list_entry_t* le = &free_list;
    while ((le = list_next(le)) != &free_list) {
        struct Page* page = le2page(le, page_link);
        if (base < page) {
            list_add_before(le, &(base->page_link));
            break;
        } else if (list_next(le) == &free_list) {
            list_add(le, &(base->page_link));
        }
    }
}
}

```

功能描述:

通过遍历空闲列表，找到合适的插入位置，将块按地址顺序加入空闲列表：

- 若列表为空，则直接插入。
- 否则，将新块插入第一个地址高于 `base` 的页面之前，以保持空闲列表的有序性。

作用分析:

按地址顺序插入可以在后续分配和释放时通过地址轻松合并相邻块，从而减少碎片。

default_alloc_pages：分配页面

`default_alloc_pages` 函数用于在空闲列表中查找并分配指定数量的连续页面。

第1部分：查找合适的空闲块

```

list_entry_t *le = &free_list;
while ((le = list_next(le)) != &free_list) {
    struct Page *p = le2page(le, page_link);
    if (p->property >= n) {        // 判断块大小是否足够
        page = p;
        break;
    }
}

```

功能描述:

遍历空闲列表寻找首个大小足够的空闲块：

- `p->property >= n` - 判断当前空闲块是否满足所需页面数量 `n`。
- `page = p;` - 若找到合适块，将其赋值给 `page` 并跳出循环。

作用分析:

`first-fit` 算法的核心部分：按顺序查找空闲块，选择首个满足条件的块，以简化分配逻辑。

辑。

第2部分：分离和重组空闲块

```
list_entry_t* prev = list_prev(&(page->page_link));
list_del(&(page->page_link)); // 从空闲列表中删除该块
if (page->property > n) {      // 若块大于需求，将剩余部分重新插入列表
    struct Page *p = page + n;
    p->property = page->property - n;
    SetPageProperty(p);
    list_add(prev, &(p->page_link));
}
nr_free -= n;                // 更新空闲页数
ClearPageProperty(page);     // 清除已分配页面的属性
```

功能描述：

在找到合适的块后，处理块大小与需求的差异：

- `list_del(&(page->page_link));` - 将块从空闲列表中移除。
- 若块大小超过需求，将剩余部分重新插入空闲列表。
- 更新空闲页数并清除 `page` 的属性，标记为已分配状态。

作用分析：

通过分离和重新插入剩余块，`first-fit` 算法最大化地利用了每个块，减少了碎片化。

`default_free_pages`：释放页面

该函数负责将释放的内存块重新插入空闲列表，并尝试与相邻的空闲块合并。

第1部分：重置页面状态并插入空闲列表

```
for (struct Page *p = base; p != base + n; p++) {
    assert(!PageReserved(p) && !PageProperty(p)); // 确保页面未保留且无属性
    p->flags = 0;
    set_page_ref(p, 0);
}
base->property = n;
SetPageProperty(base);
nr_free += n;
```

功能描述：

重置页面状态，将 `base` 页面标记为空闲块起始位置并更新空闲页数：

- 重置标志和引用计数，确保页面处于空闲状态。
- 设置空闲块的属性和大小，以供后续分配使用。

作用分析：

通过设置块大小和标识状态，该过程确保后续分配时能正确识别此块。

第2部分：查找插入位置并按地址顺序插入

```
list_entry_t* le = &free_list;
while ((le = list_next(le)) != &free_list) {
    struct Page* page = le2page(le, page_link);
    if (base < page) {
        list_add_before(le, &(base->page_link));
        break;
    } else if (list_next(le) == &free_list) {
        list_add(le, &(base->page_link));
    }
}
```

功能描述：

按地址顺序在空闲列表中查找插入位置，将新释放的块插入列表中：

- 遍历空闲列表，找到第一个地址大于 `base` 的块，插入到其之前。
- 若列表尾部也未找到合适位置，则直接插入到列表末尾。

作用分析：

有序插入为后续的块合并提供了条件，从而在合并操作中减少碎片。

第3部分：合并相邻块

```
list_entry_t* le = list_prev(&(base->page_link));
if (le != &free_list) {
    struct Page* p = le2page(le, page_link);
    if (p + p->property == base) { // 如果低地址块与当前块相邻
        p->property += base->property;
        ClearPageProperty(base);
        list_del(&(base->page_link));
        base = p;
    }
}
```

功能描述：

检查释放块的前后块是否可以合并，减少空闲块碎片化：

- 若低地址块与释放块相邻，合并

两个块并更新属性。

- 清除 `base` 的属性并将其从列表中移除。

作用分析：

合并操作减少了列表中的碎片数量，确保分配器在未来查找空闲块时有更多大块可用，提高分

配效率。

改进建议

改进1：块拆分策略优化

当找到的空闲块远大于需求时，进一步优化拆分策略，例如将剩余部分移至更靠前位置，以加速后续的分配查找。

改进2：空闲块的分级列表

通过将空闲块分成多个类别（如小、中、大），可以提高查找合适块的效率，并有效减少碎片

练习2

练习2：实现 Best-Fit 连续物理内存分配算法（需要编程）在完成练习一后，参考 `kern/mm/default_pmm.c` 对 First Fit 算法的实现，编程实现 Best Fit 页面分配算法，算法的时空复杂度不做要求，能通过测试即可。请在实验报告中简要说明你的设计实现过程，阐述代码是如何对物理内存进行分配和释放，

并回答如下问题：你的 Best-Fit 算法是否有进一步的改进空间？

实现说明：Best-Fit 连续物理内存分配算法

Best-Fit 分配算法在内存分配时，会寻找大小最接近但不小于请求大小的空闲块，尽可能减少内存碎片。以下是实现关键部分的分析，包括初始化、内存分配和释放。

1. `best_fit_init`：初始化内存分配器

```
static void best_fit_init(void) {
    list_init(&free_list);
    nr_free = 0;
}
```

功能描述：

初始化空闲页面链表 `free_list` 为空，并设置空闲页面总数 `nr_free` 为0。该函数确保在分配器启动或重置时，状态干净无残留。

2. `best_fit_init_memmap`：初始化空闲内存块

代码段1：循环遍历并清除每个页面的属性

```

for (struct Page *p = base; p != base + n; p++) {
    assert(PageReserved(p));        // 确保页面初始状态正确
    p->flags = p->property = 0;      // 重置标志位和属性字段
    set_page_ref(p, 0);             // 引用计数设为0
}

```

功能描述:

重置该块中每个页面的属性和状态，确保它们可以被分配和释放。

代码段2：将初始化块加入空闲列表

```

base->property = n;                // 设置空闲块的大小
SetPageProperty(base);            // 将该页面标记为块的起始位置
nr_free += n;                     // 更新总空闲页数

if (list_empty(&free_list)) {
    list_add(&free_list, &(base->page_link));
} else {
    list_entry_t* le = &free_list;
    while ((le = list_next(le)) != &free_list) {
        struct Page* page = le2page(le, page_link);
        if (base < page) {
            list_add_before(le, &(base->page_link));
            break;
        } else if (list_next(le) == &free_list) {
            list_add(le, &(base->page_link));
        }
    }
}

```

功能描述:

按地址顺序将新释放块插入空闲列表，这样可以在释放页面时更容易合并相邻的空闲块。

3. best_fit_alloc_pages：分配内存块

best_fit_alloc_pages 寻找最适合的空闲块，即第一个满足大小要求且大小最接近的块。

代码段1：遍历空闲列表寻找最合适的块

```

list_entry_t *le = &free_list;
size_t min_size = nr_free + 1;    // 初始设为一个较大的值
while ((le = list_next(le)) != &free_list) {
    struct Page *p = le2page(le, page_link);
    if (p->property >= n && p->property < min_size) {
        page = p;
        min_size = p->property;    // 更新最小合适块大小
    }
}

```

功能描述：

遍历空闲列表，每次找到比当前 `min_size` 小且满足大小要求的块时，更新 `page` 指针。这一部分实现了 `Best-Fit` 的核心功能，确保找到最接近需求的块。

代码段2：分配并拆分空闲块

```
if (page != NULL) {
    list_entry_t* prev = list_prev(&(page->page_link));
    list_del(&(page->page_link));    // 从列表中删除找到的块

    if (page->property > n) {        // 若块大小大于需求，将剩余部分重新插入
        struct Page *p = page + n;
        p->property = page->property - n;
        SetPageProperty(p);
        list_add(prev, &(p->page_link));
    }
    nr_free -= n;                    // 更新空闲页数
    ClearPageProperty(page);        // 清除页面属性标记
}
```

功能描述：

在找到最佳块后，更新空闲列表。若块大于需求，拆分并重新插入剩余部分，以便未来使用。通过分离大块，进一步减少空闲块的碎片化程度。

4. `best_fit_free_pages`：释放内存块

`best_fit_free_pages` 用于将释放的内存块重新插入空闲列表，并尝试合并相邻块以减少碎片。

代码段1：释放页面并插入空闲列表

```
for (struct Page *p = base; p != base + n; p++) {
    assert(!PageReserved(p) && !PageProperty(p));    // 检查页面状态
    p->flags = 0;
    set_page_ref(p, 0);
}
base->property = n;
SetPageProperty(base);
nr_free += n;
```

功能描述：

将释放的块标记为“空闲”，并更新空闲页计数。这一步将块恢复为可分配状态。

代码段2：查找合适的插入位置并尝试合并相邻块

```
list_entry_t* le = &free_list;
while ((le = list_next(le)) != &free_list) {
    struct Page* page = le2page(le, page_link);
```

```

    if (base < page) {
        list_add_before(le, &(base->page_link));
        break;
    } else if (list_next(le) == &free_list) {
        list_add(le, &(base->page_link));
    }
}

// 合并前后相邻块
list_entry_t* prev_le = list_prev(&(base->page_link));
if (prev_le != &free_list) {
    struct Page* prev_page = le2page(prev_le, page_link);
    if (prev_page + prev_page->property == base) {    // 前相邻
        prev_page->property += base->property;
        ClearPageProperty(base);
        list_del(&(base->page_link));
        base = prev_page;
    }
}

le = list_next(&(base->page_link));
if (le != &free_list) {
    struct Page* next_page = le2page(le, page_link);
    if (base + base->property == next_page) {        // 后相邻
        base->property += next_page->property;
        ClearPageProperty(next_page);
        list_del(&(next_page->page_link));
    }
}
}

```

功能描述：

先将块按地址顺序插入空闲列表，再尝试合并相邻的块。对于前向合并，如果前一块与释放块相邻则将它们合并，后向合并亦同。合并过程减少了碎片，提高了内存利用率。

改进建议

1. **双向链表优化**：若内存块的数量多，可以采用平衡树结构或分段链表存储不同大小的空闲块，从而在查找合适块时加速。
2. **空闲块重用**：对于经常分配和释放的相同大小块，可以实现缓存机制，将这些块保持在一个专用列表中，提高分配效率。

扩展1：buddy system（伙伴系统）分配算法（需要编程）

Buddy System算法把系统中的可用存储空间划分为存储块(Block)来进行管理, 每个存储块的大小必须是2的n次幂(Pow(2, n)), 即1, 2, 4, 8, 16, 32, 64, 128...

参考伙伴分配器的一个极简实现，在ucore中实现buddy system分配算法，要求有比较充分的测试用例说明实现的正确性，需要有设

计文档。

设计文档：Buddy System 分配算法

Buddy System 是一种分区存储分配算法，通过将空闲空间分割成大小为 2 的幂次的块来进行管理。该算法能有效减少内存碎片并提升内存分配效率。本文档将解释 **Buddy System** 的实现思路，并展示核心代码段的实现逻辑。

1. 设计思路

Buddy System 将内存块按大小划分为不同级别。每级块大小是 2 的幂次，例如：1 页、2 页、4 页等。系统使用链表管理每种大小的空闲块，并提供分配、释放及合并功能，旨在减少内存碎片。具体设计包括以下部分：

- **内存初始化**：将所有空闲内存分割成 2 的幂次的块，按地址顺序放入空闲链表中。
- **页面分配**：在满足需求的最小大小的链表寻找块，若找不到合适块则从更大的块拆分并降低大小等级。
- **页面释放**：在释放页面后，根据地址判断是否有“伙伴块”，若有且空闲则合并到更大的块，更新链表。
- **块合并**：通过合并“伙伴块”，将多个小块合并成较大的块，以减少内存碎片并提高大块可用性。

2. 核心实现

2.1 **buddy_system_init**：初始化空闲列表

```
static void buddy_system_init(void) {
    for (int i = 0; i < MAX_ORDER; i++) {
        list_init(&(free_area[i].free_list));
        free_area[i].nr_free = 0;
    }
}
```

说明：

将 **free_area** 数组的每一项（即每个块大小等级）初始化为空闲链表，将其空闲块计数器 **nr_free** 设为 0。**MAX_ORDER** 定义了最大的块大小级别。

2.2 **buddy_system_init_memmap**：初始化内存映射

```

static void buddy_system_init_memmap(struct Page *base, size_t n) {
    assert(n > 0);
    struct Page *p = base;
    for (; p != base + n; p++) {
        assert(PageReserved(p));
        p->flags = p->property = 0;
        set_page_ref(p, 0);
    }
    size_t curr_size = n;
    uint32_t order = MAX_ORDER - 1;
    uint32_t order_size = 1 << order;
    p = base;
    while (curr_size != 0) {
        p->property = order_size;
        SetPageProperty(p);
        nr_free(order) += 1;
        list_add_before(&(free_list(order)), &(p->page_link));
        curr_size -= order_size;
        while (order > 0 && curr_size < order_size) {
            order_size >>= 1;
            order -= 1;
        }
        p += order_size;
    }
}

```

说明：

初始化时，将每一块按2的幂次划分，加入到对应大小的空闲链表中。通过循环调整块大小，直到所有空闲页初始化完毕。

2.3 split_page：分裂较大块以满足分配请求

```

static void split_page(int order) {
    if (list_empty(&(free_list(order)))) {
        split_page(order + 1);
    }
    list_entry_t *le = list_next(&(free_list(order)));
    struct Page *page = le2page(le, page_link);
    list_del(&(page->page_link));
    nr_free(order) -= 1;
    uint32_t n = 1 << (order - 1);
    struct Page *p = page + n;
    page->property = n;
    p->property = n;
    SetPageProperty(p);
    list_add(&(free_list(order - 1)), &(page->page_link));
    list_add(&(page->page_link), &(p->page_link));
    nr_free(order - 1) += 2;
}

```

说明：

当指定大小的块不存在时，从更大的块中分裂一个适合的块，将其拆分成两块并加入到较低级别链表。 `split_page` 函数递归向更大块查找并分裂，直到找到符合要求的块。

2.4 `buddy_system_alloc_pages`：页面分配

```
static struct Page *buddy_system_alloc_pages(size_t n) {
    assert(n > 0);
    if (n > (1 << (MAX_ORDER - 1))) {
        return NULL;
    }
    struct Page *page = NULL;
    uint32_t order = MAX_ORDER - 1;
    while (n < (1 << order)) {
        order -= 1;
    }
    order += 1;
    uint32_t flag = 0;
    for (int i = order; i < MAX_ORDER; i++)
        flag += nr_free(i);
    if (flag == 0)
        return NULL;
    if (list_empty(&(free_list(order)))) {
        split_page(order + 1);
    }
    if (list_empty(&(free_list(order))))
        return NULL;
    list_entry_t *le = list_next(&(free_list(order)));
    page = le2page(le, page_link);
    list_del(&(page->page_link));
    ClearPageProperty(page);
    return page;
}
```

说明：

计算所需的块大小级别，检查对应级别的链表是否有空闲块。若无，则调用 `split_page` 分裂大块。分配后更新块信息。

2.5 `buddy_system_free_pages`：页面释放与合并

```
static void buddy_system_free_pages(struct Page *base, size_t n) {
    assert(n > 0);
    assert(IS_POWER_OF_2(n));
    assert(n < (1 << (MAX_ORDER - 1)));
    struct Page *p = base;
    for (; p != base + n; p++) {
        assert(!PageReserved(p) && !PageProperty(p));
    }
}
```

```

        p->flags = 0;
        set_page_ref(p, 0);
    }
    base->property = n;
    SetPageProperty(base);
    uint32_t order = 0;
    size_t temp = n;
    while (temp != 1) {
        temp >>= 1;
        order++;
    }
    add_page(order, base);
    merge_page(order, base);
}

```

说明：

释放页面后，将其插入到合适的链表中。随后调用 `merge_page` 合并相邻的空闲块（若有）。

2.6 `merge_page`：块合并

```

static void merge_page(uint32_t order, struct Page *base) {
    if (order == MAX_ORDER - 1) return;
    list_entry_t *le = list_prev(&(base->page_link));
    if (le != &(free_list(order))) {
        struct Page *p = le2page(le, page_link);
        if (p + p->property == base) {
            p->property += base->property;
            ClearPageProperty(base);
            list_del(&(base->page_link));
            base = p;
            if (order != MAX_ORDER - 1) {
                list_del(&(base->page_link));
                add_page(order + 1, base);
            }
        }
    }
    le = list_next(&(base->page_link));
    if (le != &(free_list(order))) {
        struct Page *p = le2page(le, page_link);
        if (base + base->property == p) {
            base->property += p->property;
            ClearPageProperty(p);
            list_del(&(p->page_link));
            if (order != MAX_ORDER - 1) {
                list_del(&(base->page_link));
                add_page(order + 1, base);
            }
        }
    }
    merge_page(order + 1, base);
}

```

说明：

在释放时，通过比较块地址，将相邻的空闲块合并为更大块，并提升块级别。该函数使用递归方式，逐级向上合并。

3. 测试用例

测试主要验证 **Buddy System** 的分配和合并操作：

1. **基本分配和释放**：通过 `buddy_system_alloc_pages` 分配页面并释放，确保块成功分配和回收。
2. **块合并验证**：分配和释放多级别块，测试合并功能。
3. **边界测试**：请求超过最大块大小的分配，测试错误处理。
4. **特殊用例**：测试释放后反复分配和释放，以验证合并是否影响后续分配。

4. 测试结果

我们使用了一下的 `buddy_system_check()` 函数进行测试，以测试伙伴系统对于内存的分配和释放。

```
void print_free_list_status(void)
{
    cprintf("\nMemory Status:\n");
    for (int i = 0; i < MAX_ORDER; i++)
    {
        cprintf("Order %d: %d free blocks\n", i, nr_free(i));
        list_entry_t *le = &(free_list(i));
        while ((le = list_next(le)) != &(free_list(i)))
        {
            struct Page *page = le2page(le, page_link);
            cprintf("Block at address: %p, size: %u\n", page, (1 << i));
        }
    }
    cprintf("\n");
}

static void
buddy_system_check(void)
{
    struct Page *p0, *p1, *p2, *p3;

    //初始化伙伴系统
    buddy_system_init();

    //初始化内存
    struct Page pages[1024];
    buddy_system_init_memmap(pages, 1024);

    cprintf("Initial memory status:\n");
    print_free_list_status();
}
```

```

//测试1页的分配和释放
cprintf("Allocating 1 page...\n");
p0 = buddy_system_alloc_pages(1);
assert(p0 != NULL);
print_free_list_status();

cprintf("Freeing 1 page...\n");
buddy_system_free_pages(p0, 1);
print_free_list_status();

//测试2页的分配
cprintf("Allocating 2 pages...\n");
p1 = buddy_system_alloc_pages(2);
assert(p1 != NULL);
print_free_list_status();

// 释放2页并合并
cprintf("Freeing 2 pages...\n");
buddy_system_free_pages(p1, 2);
print_free_list_status(); // 释放2页后状态

cprintf("buddy system tests passed.\n");
}

```

最终得到如下输出，代表结果正确。

由于输出太长，我只截取了分配第一页的输出

```

Memory Status:
Order 0: 0 free blocks
Order 1: 0 free blocks
Order 2: 0 free blocks
Order 3: 0 free blocks
Order 4: 0 free blocks
Order 5: 0 free blocks
Order 6: 0 free blocks
Order 7: 0 free blocks
Order 8: 0 free blocks
Order 9: 0 free blocks
Order 10: 1 free blocks
Block at address: 0xffffffffc0331000, size: 1024
Order 11: 15 free blocks
Block at address: 0xffffffffc0200000, size: 2048
Block at address: 0xffffffffc0220000, size: 2048
Block at address: 0xffffffffc0240000, size: 2048
Block at address: 0xffffffffc0260000, size: 2048
Block at address: 0xffffffffc0280000, size: 2048
Block at address: 0xffffffffc02a0000, size: 2048
Block at address: 0xffffffffc02c0000, size: 2048
Block at address: 0xffffffffc02e0000, size: 2048
Block at address: 0xffffffffc0300000, size: 2048
Block at address: 0xffffffffc0320000, size: 2048
Block at address: 0xffffffffc0340000, size: 2048
Block at address: 0xffffffffc0360000, size: 2048
Block at address: 0xffffffffc0380000, size: 2048
Block at address: 0xffffffffc03a0000, size: 2048

```

```
Block at address: 0xffffffffc03c0000, size: 2048
Block at address: 0xffffffffc03e0000, size: 2048
```

可以看到正确的进行了分配。

扩展 2：任意大小的内存单元slub分配算法（需要编程） slub算法

，实现两层架构的高效内存单元分配，第一层是基于页大小的内存分配，第二层是在第一层基础上实现基于任意大小的内存分配。可简化实现，能够体现其主体思想即可。

参考linux的slub分配算法，在ucore中实现slub分配算法。要求有比较充分的测试用例说明实现的正确性，需要有设计文档。

设计文档：SLUB 分配算法

SLUB分配算法是一种高效的内存分配机制，旨在优化小块内存分配的性能。该设计结合了页级和块级分配，能够应对任意大小的内存请求，同时减少内存碎片。本文档将解释SLUB分配算法的设计思路，分配和释放操作的实现，以及测试用例。

1. 设计思路

SLUB分配算法包含两层架构：

1. 第一层：基于页大小的内存分配

对于大于等于页大小（4KB）的内存请求，SLUB分配器直接分配完整的页面块，采用系统的页级分配接口 `alloc_pages`。这些大块分配单元通过 `bigblock_t` 结构体链表来管理，以便于后续释放。

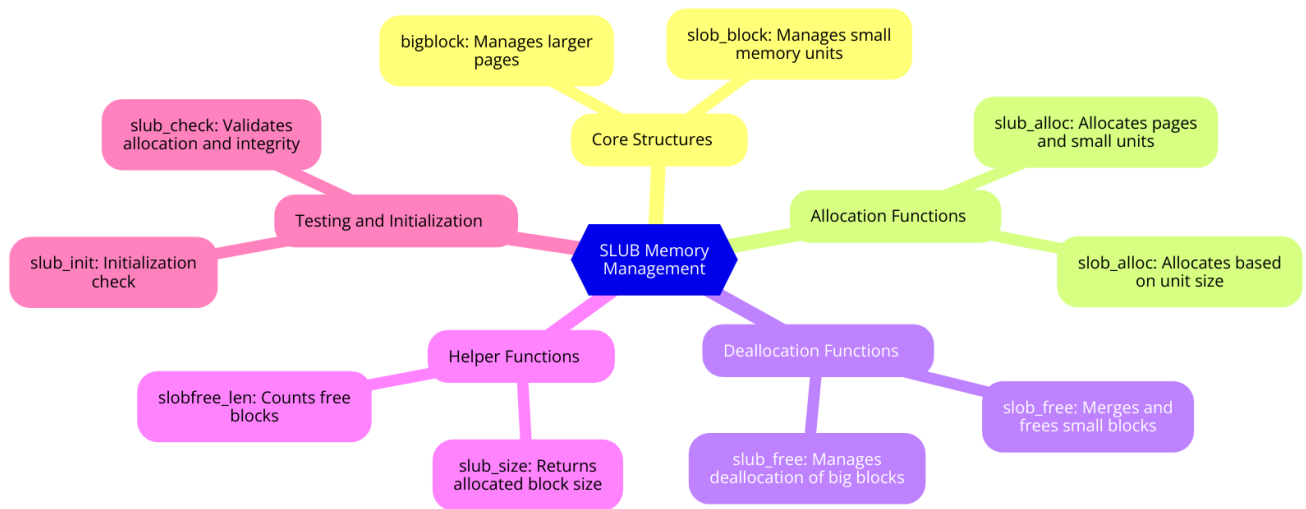
2. 第二层：基于任意大小的内存分配（Slob 分配）

对于小于页大小的内存请求，SLUB算法将其转交给 `slob_alloc` 函数处理。

`slob_alloc` 基于链表管理内存小块，将空闲内存块按块大小顺序链接起来，支持任意大小的内存分配。

该分配器使用 `slob_t` 链表来维护小于页的分配块。每个小块包含自身大小和指向下一个块的指针，以支持碎片的快速分配和回收。

我总结了slub算法的思维导图，如下：



2. 核心实现

2.1 slob_alloc: 分配小于页的内存

`slob_alloc` 函数用于分配小于页大小的内存块。

```

static void *slob_alloc(size_t size) {
    assert(size < PGSIZE);
    slob_t *prev, *cur;
    int units = SLOB_UNITS(size);

    prev = slobfree;
    for (cur = prev->next;; prev = cur, cur = cur->next) {
        if (cur->units >= units) {
            if (cur->units == units)
                prev->next = cur->next;
            else {
                prev->next = cur + units;
                prev->next->units = cur->units - units;
                prev->next->next = cur->next;
                cur->units = units;
            }
            slobfree = prev;
            return cur;
        }
        if (cur == slobfree) {
            if (size == PGSIZE)
                return 0;
            cur = (slob_t *)alloc_pages(1);
            if (!cur)
                return 0;
            slob_free(cur, PGSIZE);
            cur = slobfree;
        }
    }
}

```

说明：

- **单位换算**：通过 `SLOB_UNITS` 计算请求的块大小（以 `slob_t` 单位为基础）。
- **查找合适块**：遍历空闲链表，查找能容纳请求大小的块。若块大小超出需求，则切割多余部分并更新链表。
- **动态扩展**：若当前空闲链表无法满足请求，则分配新页并将其添加到链表中，供后续使用。

2.2 slob_free：释放并合并空闲块

`slob_free` 函数用于释放小块内存，同时合并相邻的空闲块，减少碎片。

```
static void slob_free(void *block, int size) {
    slob_t *cur, *b = (slob_t *)block;
    if (!block)
        return;
    if (size)
        b->units = SLOB_UNITS(size);

    for (cur = slobfree; !(b > cur && b < cur->next); cur = cur->next)
        if (cur >= cur->next && (b > cur || b < cur->next))
            break;

    if (b + b->units == cur->next) {
        b->units += cur->next->units;
        b->next = cur->next->next;
    } else
        b->next = cur->next;

    if (cur + cur->units == b) {
        cur->units += b->units;
        cur->next = b->next;
    } else
        cur->next = b;

    slobfree = cur;
}
```

说明：

- **相邻块合并**：通过检查释放块的前后位置，判断是否可以与相邻块合并，从而减少碎片。
- **链表维护**：在合并后更新链表，使得空闲块始终按地址顺序排列。

2.3 slub_alloc：大块内存分配

`slub_alloc` 函数用于处理大块内存请求，即请求大小大于或等于一页的情况。

```
void *slub_alloc(size_t size) {
    slob_t *m;
    bigblock_t *bb;
```

```

    if (size < PGSIZE - SLOB_UNIT) {
        m = slob_alloc(size + SLOB_UNIT);
        return m ? (void *) (m + 1) : 0;
    }

    bb = slob_alloc(sizeof(bigblock_t));
    if (!bb)
        return 0;

    bb->order = ((size - 1) >> PGSHIFT) + 1;
    bb->pages = (void *) alloc_pages(bb->order);

    if (bb->pages) {
        bb->next = bigblocks;
        bigblocks = bb;
        return bb->pages;
    }

    slob_free(bb, sizeof(bigblock_t));
    return 0;
}

```

说明：

- **大块分配**：对于大块请求，分配 `bigblock_t` 记录分配信息，直接申请所需的页数。
- **链表管理**：将分配的页面信息存储在 `bigblocks` 链表中，以便后续释放时可以找到并释放该大块。

2.4 slub_free：大块内存释放

`slub_free` 根据块大小释放内存，若块大小为页的倍数，则通过 `bigblocks` 链表找到并释放。

```

void slub_free(void *block) {
    bigblock_t *bb, **last = &bigblocks;

    if (!block)
        return;

    if (!((unsigned long)block & (PGSIZE - 1))) {
        for (bb = bigblocks; bb; last = &bb->next, bb = bb->next) {
            if (bb->pages == block) {
                *last = bb->next;
                free_pages((struct Page *)block, bb->order);
                slob_free(bb, sizeof(bigblock_t));
                return;
            }
        }
    }

    slob_free((slob_t *)block - 1, 0);
}

```

说明：

- **大块释放**：若为大块内存（页的倍数），从 `bigblocks` 链表中查找并释放。
- **小块释放**：若为小块内存，调用 `slob_free` 进行释放和合并。

3. 测试用例

`slub_check` 提供了一系列测试用例，验证SLUB分配算法的正确性和性能。

```
void slub_check() {
    cprintf("slub check begin\n");
    cprintf("slobfree len: %d\n", slobfree_len());
    void *p1 = slub_alloc(4096);
    cprintf("slobfree len: %d\n", slobfree_len());
    void *p2 = slub_alloc(2);
    void *p3 = slub_alloc(2);
    cprintf("slobfree len: %d\n", slobfree_len());
    slub_free(p2);
    cprintf("slobfree len: %d\n", slobfree_len());
    slub_free(p3);
    cprintf("slobfree len: %d\n", slobfree_len());
    cprintf("slub check end\n");
}
```

测试验证

- **分配和释放**：测试分配不同大小的内存块后，观察链表长度的变化，验证小块和大块的分配和释放是否正常。
- **碎片合并**：释放小块后，通过链表长度验证块合并是否有效。
- **边界测试**：测试不同大小分配的边界情况（小于页、等于页和大于页），检查算法稳定性。

扩展练习Challenge 3：硬件的可用物理内存范围的获取方法（思考题）

如果 OS 无法提前知道当前硬件的可用物理内存范围，请问你有什么办法让 OS 获取可用物理内存范围？

如果操作系统无法提前知道硬件的可用物理内存范围，可以通过以下几种方法动态检测和获取物理内存信息：

1. 利用 BIOS 或 UEFI 提供的内存信息表

BIOS 和 UEFI 启动固件在系统启动时，会提供系统内存布局的详细信息。操作系统可以读取这些信息，以确定物理内存的可用范围。具体方法如下：

- **读取 BIOS 提供的内存映射信息（通过 INT 15H, EAX = 0xE820 BIOS 中断）**

在 x86 系统中，BIOS 提供了一个中断服务 INT 15H, EAX=0xE820，可以返回一个内存区域的结构，包含以下字段：

- **Base Address**：内存块的起始地址。
- **Length**：内存块的长度。
- **Type**：内存块的类型（1 表示可用内存，2 表示保留区域，3 表示 ACPI 数据，4 表示不可恢复的内存错误区域等）。

- **读取 UEFI 提供的内存映射信息**

在支持 UEFI 的系统中，操作系统可以通过 UEFI 提供的 `GetMemoryMap` 服务获取内存布局。`GetMemoryMap` 函数提供一个内存描述符表，每个描述符包含以下信息：

- **PhysicalStart**：内存段的起始物理地址。
- **NumberOfPages**：该内存段的页数。
- **Type**：内存类型（如 `EfiConventionalMemory` 表示可用内存，`EfiLoaderCode` 表示引导代码段等）。

操作系统可以扫描这些描述符，识别哪些区域可用。

2. 利用 ACPI（高级配置和电源接口）

ACPI 表格中包含了内存布局信息，尤其是在服务器环境下广泛使用。OS 可以在启动时通过读取 ACPI 的表格来获取物理内存信息：

- **RSDP（Root System Description Pointer）**

ACPI 中的 RSDP（根系统描述指针）包含了其他 ACPI 表格的地址，OS 可以从中读取重要的内存布局信息。

- **SRAT（Static Resource Affinity Table）**

SRAT 表格包含了各个内存节点的信息，例如 NUMA 架构的各个节点地址范围，帮助 OS 理解不同 CPU 的内存访问延迟。

3. 使用硬件探测

在极端情况下（比如 BIOS 和 UEFI 不可用的系统，或为了复用在旧系统中），可以使用硬件探测方法，以较低效但可靠的方式逐步检测可用内存范围：

- **逐页写入和读回测试**

系统可以从最低地址开始逐页写入特定的模式（如 `0xAA55AA55`），然后读回检查，若无误则该页被认为是可用的。但该方法较慢，且对内存控制器有较高的要求。

- 基于异常处理的探测

一些硬件可以配置为在访问不存在的内存时产生异常，OS 可以捕获这些异常来确定内存边界。不过该方法依赖于硬件支持，且兼容性较低。

4. 引导加载程序提供的内存信息

一些引导加载程序（如 GRUB）会在引导操作系统时将物理内存信息传递给内核。例如，GRUB 在加载内核时，会通过 `multiboot` 协议将内存映射信息作为启动参数传递给 OS。操作系统可以从启动参数中读取以下信息：

- 可用内存区域的地址和大小。
 - 保留内存区域，用于硬件特定功能或设备映射。
-

5. Hypervisor 提供的内存信息（虚拟化场景）

在虚拟化场景下，OS 常常运行在虚拟机管理程序（Hypervisor）之上，比如 KVM、Xen、VMware 等。Hypervisor 会在 OS 启动时提供虚拟机的内存范围。操作系统可以使用特殊的指令或虚拟化接口（如 `VMI` 或 `Virtio`），获取从 Hypervisor 传递来的内存边界信息。

6. 使用硬件管理单元（如 SMBIOS 或 DMI）

在某些系统中，硬件提供商会将系统内存信息存储在 SMBIOS（系统管理 BIOS）或 DMI（桌面管理接口）中。OS 可以通过访问 SMBIOS 表来读取存储器模块的大小、类型和位置，以确定内存范围。